

Lecturer: Yetmwork Tesfaye

MSc. in Computer Science and Engineering

College of Computing and Informatics

Department of Computer Science

Chapter 6: Synchronization

Objectives

At the end of this Chapter Students be able to know:-

- Overview
- Clock Synchronization
- Physical Clocks
- Logical Clocks
- Election Algorithms
- Mutual Exclusion

Introduction: What is Synchronization?

- Refers to the ordering of events and execution of instructions in time.
- Ordering of all actions
- mainly concentrate on how processes can synchronize
- Total ordering of events
 - whether message m1 from process P was sent before or after message m2 from process Q
- Ordering of access to resources
 - Shared resource(e.g Printer)

Introduction: Synchronization and Coordination

- making sure that processes doing the right thing at the right time.
- allowing processes to synchronize and coordinate their actions.
- allowing them to agree on global state (such as values of a shared variable).
- Coordinate actions and agree on values.
- Two fundamental issues:
 - Coordination (the right thing)
 - Synchronization (the right time)

Types of Synchronization

- Synchronization can be:
 - a Clock Synchronization(actual time is required):
 - **Physical clock:** concerned with “People” time.
 - **Logical Clocks**(ordering of events is required)

Types of Synchronization(Clock Synchronization)

- In a non-distributed system dealing with time is trivial as there is a single shared clock, where all processes see the same time.
- In a distributed system, on the other hand, each computer has its own clock.
- Because no clock is perfect each of these clocks has its own skew which causes clocks on different computers to drift and eventually become out of sync.
- synchronization based on actual time.

Types of Synchronization(Clock Synchronization: 1. Physical Clock Synchronization)

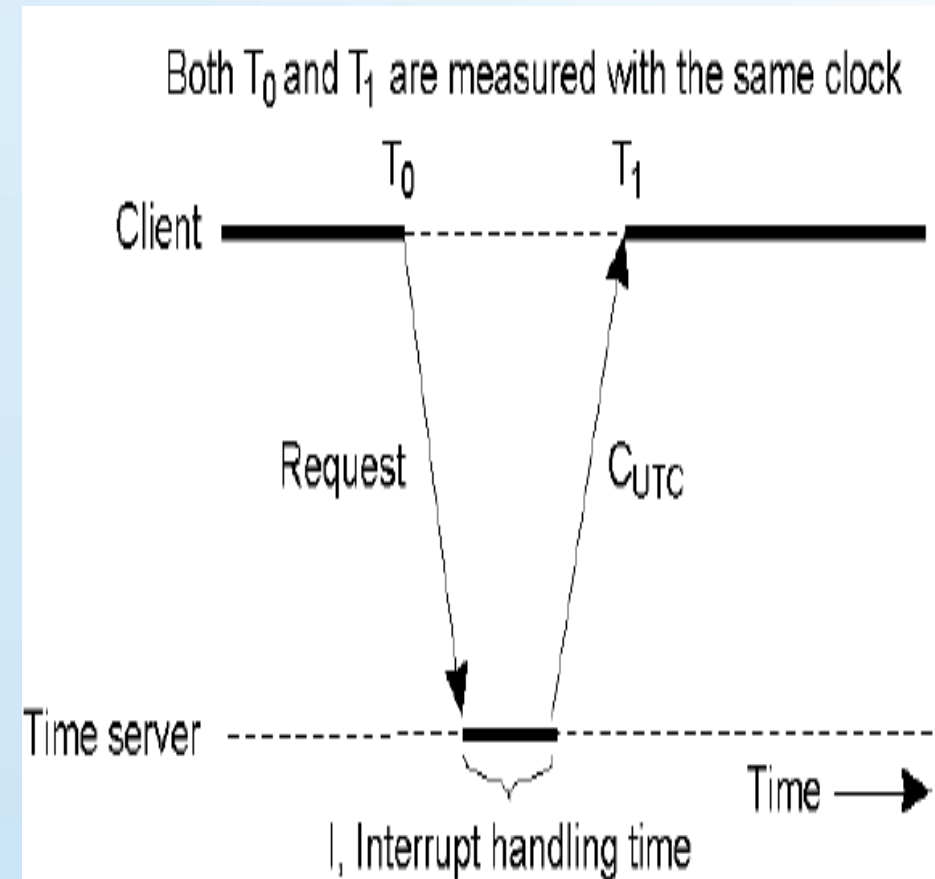
- Obtaining an Accurate Physical Time
- **A physical time server** is needed to access the current time from a universal time coordinator (UTC).
- There are two types of physical clock synchronization algorithm

1.Cristian's algorithm

2.Berkley's algorithm

Physical Clock Synchronization Algorithm (Cristian's algorithm)

- Cristian's algorithm requires clients to periodically synchronize with a central time server (typically a server with a UTC receiver)
- Every computer periodically asks the "time server" for the current time
- The server responds with the current time C_{UTC}
- The client sets its clock to C_{UTC}



Physical Clock Synchronization Algorithm (Cristian's algorithm)

- A client gets a timestamp of 4:12:30.500 from a time server. The elapsed time between the request and response was 0.020 msec. Using Cristian's algorithm, what is the time set to on the client?

- Cristian's algorithm sets the time to:

server time + $1/2$ (elapsed time)

= 4:12:30.500 + 0.020/2

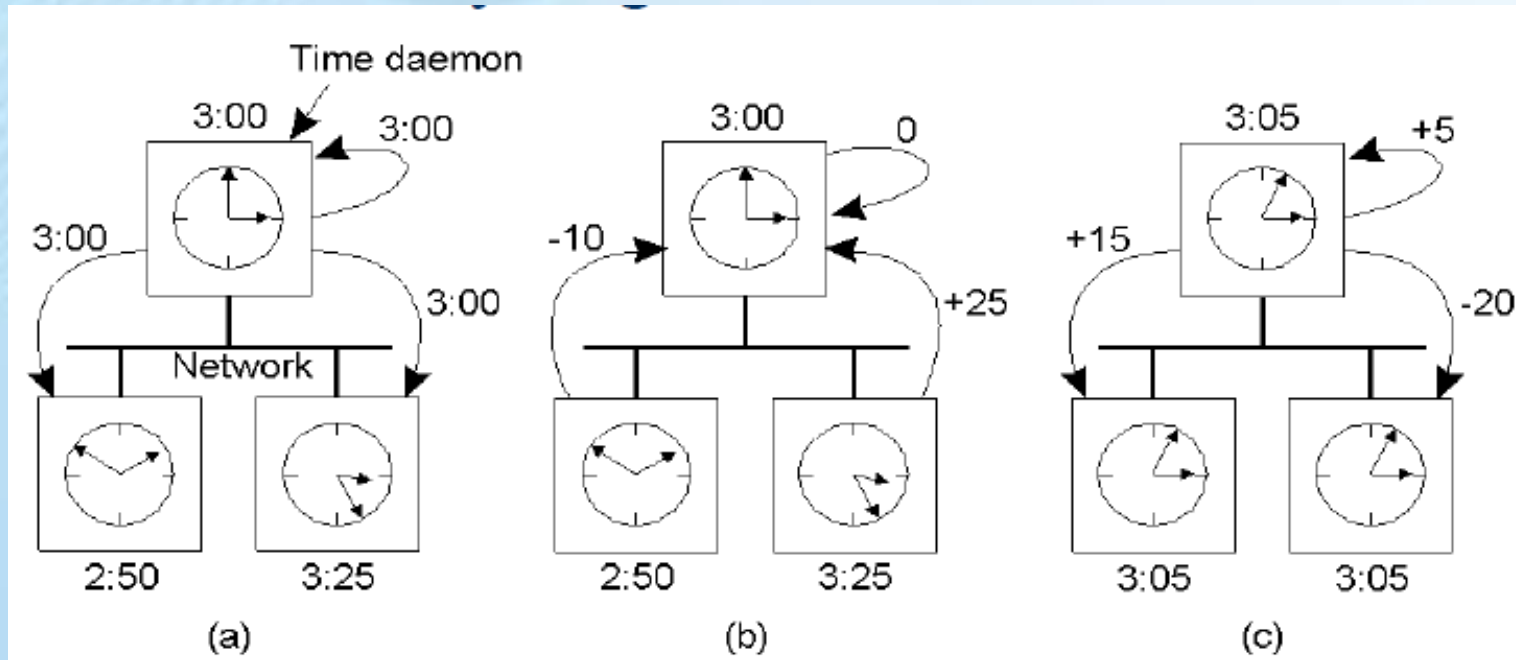
= 4:12:30.500 + 0.010

= 4:12:30.510

Physical Clock Synchronization Algorithm (Berkeley algorithm)

- An algorithm for internal synchronization of a group of computers
- A master polls to collect clock values from the others (slaves)
- It takes an average
- It sends the required adjustment to the slaves
- If master fails, can elect a new master to take over

Physical Clock Synchronization Algorithm (Berkeley Algorithm)



(a) The time daemon asks all the other machines for their clock values

(b) The machines answer

(c) Takes an average & tells everyone how to adjust their clock

Physical Clock Synchronization Algorithm (Berkeley Algorithm)

Computers	Clock Reading
A (daemon)	3:00
B (left)	2:50
C (right)	3:25

Computers	Ahead/Behind
A (daemon)	0:00
B (left)	-0:10
C (right)	+0:25

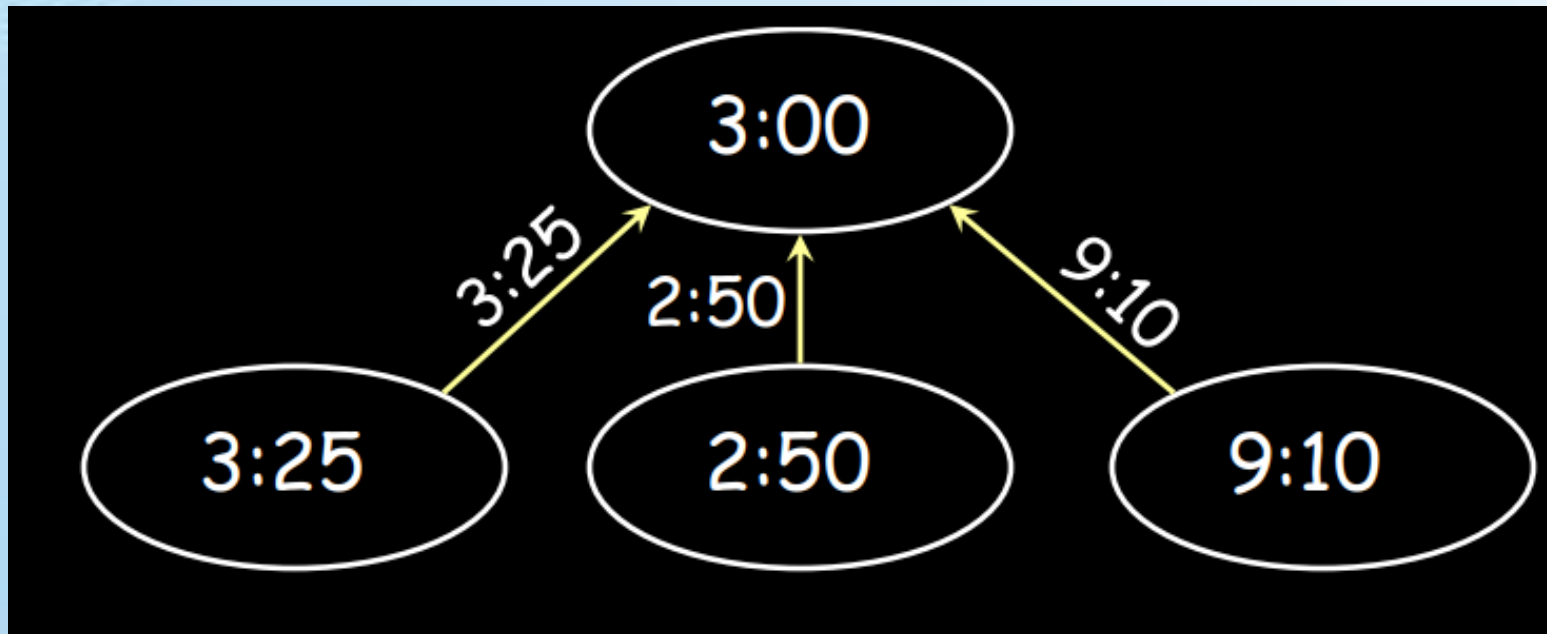
Physical Clock Synchronization Algorithm (Berkeley Algorithm)

- Average time is 3:05

Computers	Needed Adjustment
A (daemon)	+0:05
B (left)	+0:15
C (right)	-0:20

Berkeley algorithm: example

- Algorithm has provisions for ignoring readings from clocks whose skew is too great
 - Compute a fault-tolerant average

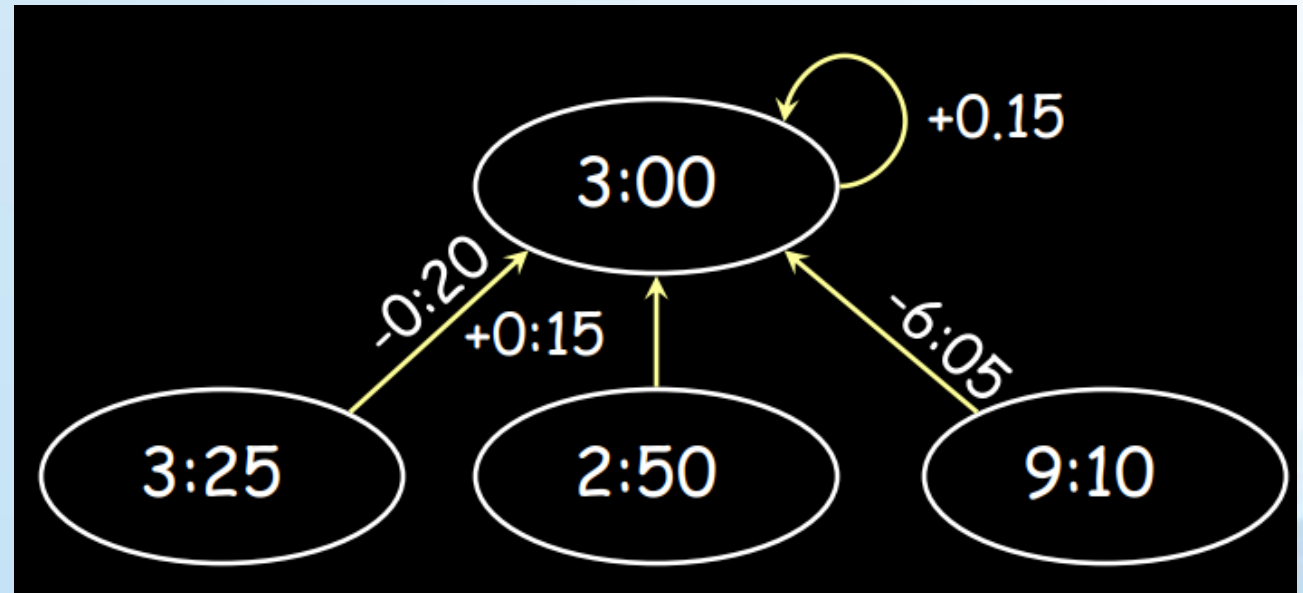


Berkeley algorithm: example

- Compute fault-tolerant average:

$$\frac{3:25 + 2:50 + 3:00}{3} = 3:05$$

Then Send offset to each client



2. Logical Clocks(what is it?)



- Refers to a mechanism for capturing chronological and casual relationship in a distributed systems.
- The relative ordering of Events is more important than actual physical time in many applications.
- Distributed Systems may have no Physically synchronous global clock.
- A logical clock allows global ordering on events from different processes in such systems.
- Logical clocks helps to order the events and identify casual relationships between them.



2. Logical Clocks

ORDERING EVENTS

- Event ordering linked with concept of **causality**:
 - Saying that event a happened before event b is same as saying that event a casually affects event b
 - If events a and b happen on processes that do not exchange any data, their exact ordering is not important.

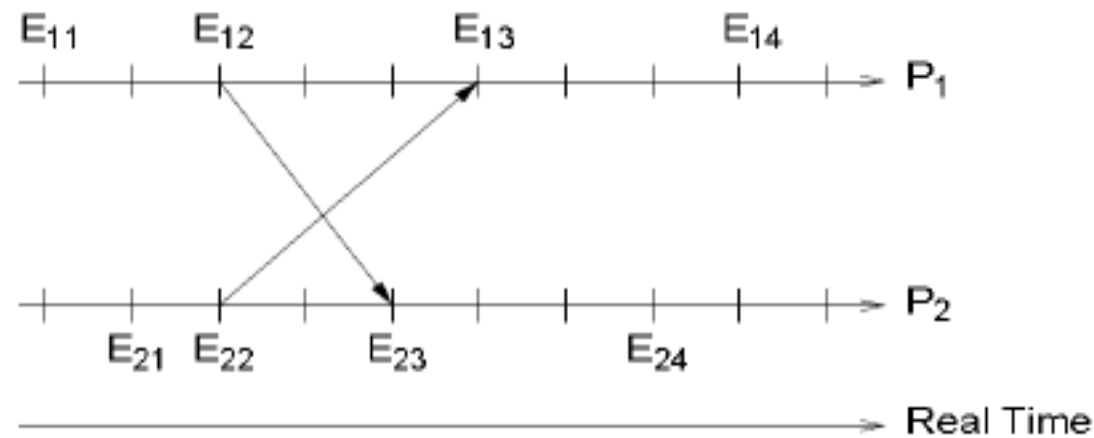
Logical Clocks: Relation “has happened before”

- Smallest relation satisfying the three conditions:
- If a and b are events in the same process and a comes before b, then $a \rightarrow b$
- If a is the sending of a message by a process and b its receipt by another process then
- $a \rightarrow b$
- If $a \rightarrow b$ and $b \rightarrow c$ then $a \rightarrow c$.
- If a did not happen before b, it cannot causally affect b(may be concurrent event).

Logical Clocks

- If $a \rightarrow b$, then a causally affects b
- We consider unordered events to be concurrent:

Example: $a \not\rightarrow b$ and $b \not\rightarrow a$ implies $a \parallel b$



- Causally related: $E_{11} \rightarrow E_{12}, E_{13}, E_{14}, E_{23}, E_{24}, \dots$
 $E_{21} \rightarrow E_{22}, E_{23}, E_{24}, E_{13}, E_{14}, \dots$
- Concurrent: $E_{11} \parallel E_{21}, E_{12} \parallel E_{22}, E_{13} \parallel E_{23}, E_{11} \parallel E_{22}, E_{13} \parallel E_{24}, \dots$

Lamport's Logical Clocks

- Each process increments local clock between any two successive events
- Message contains a timestamp
- Upon receiving a message, if received timestamp is ahead, receiver fast forward its clock to be one more than sending time.

Lamport's Logical Clocks

- Each message carries a timestamp of the senders clock.
- When a message arrives:
 - If receivers clock $<$ message timestamp
 - Set system clock to (message timestamp+1)
 - Else do nothing
- Clock must be advanced between any two events in the same process

Lamport's Logical Clocks

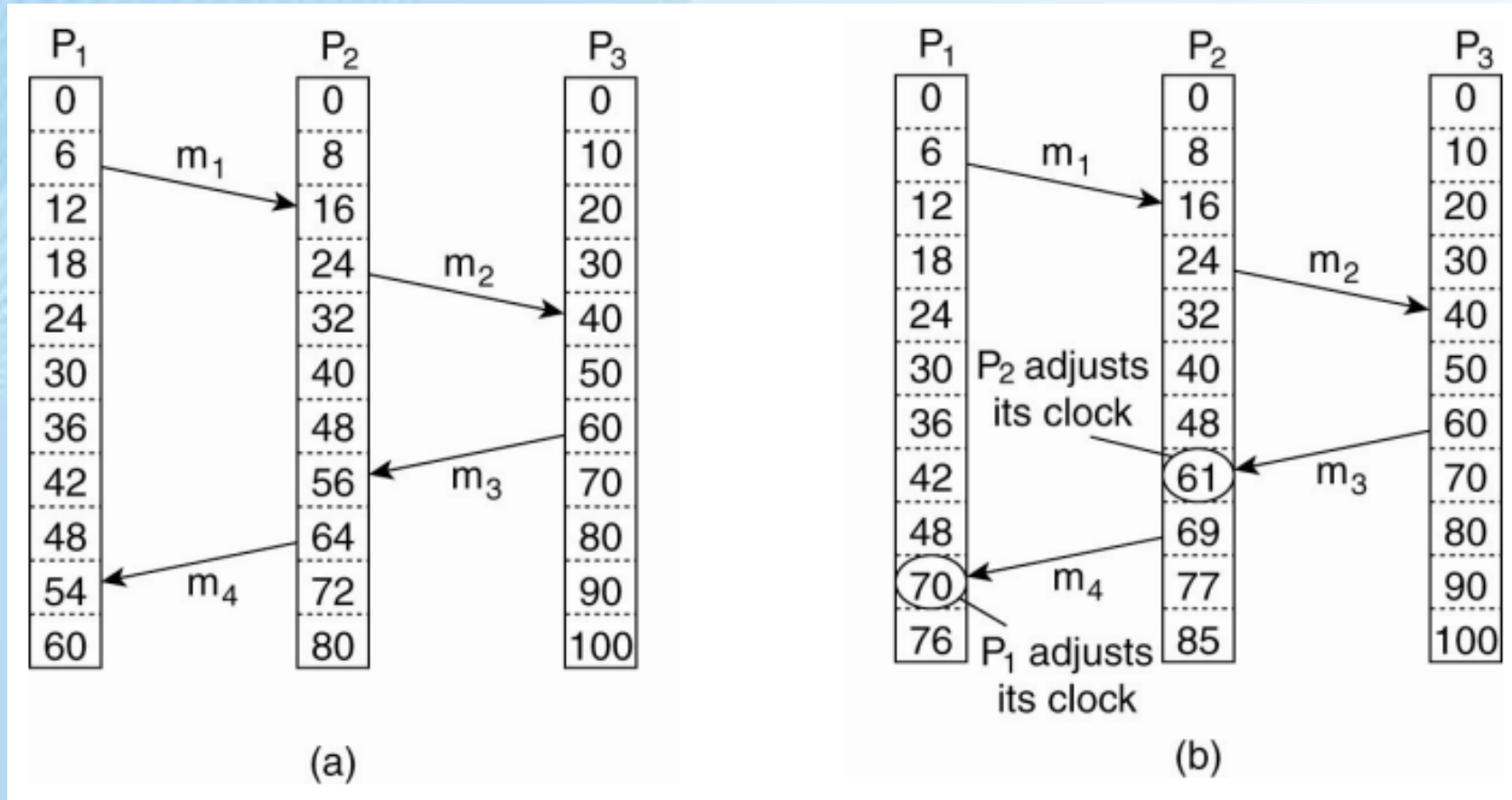
□ The algorithm for Sending:

- $\text{time} = \text{time} + 1;$
- $\text{Timestamp} = \text{time};$
- $\text{Send}(\text{message}, \text{timestamp});$

□ The algorithm for receiving:

- $(\text{message}, \text{timestamp}) = \text{receive}();$
- $\text{Time} = \max(\text{timestamp}, \text{time}) + 1;$

Lamport's Timestamp: Example



- (a) Three process, each with its own clock. The clock run at different rate
- (b) Lamport's algorithm corrects the clocks.

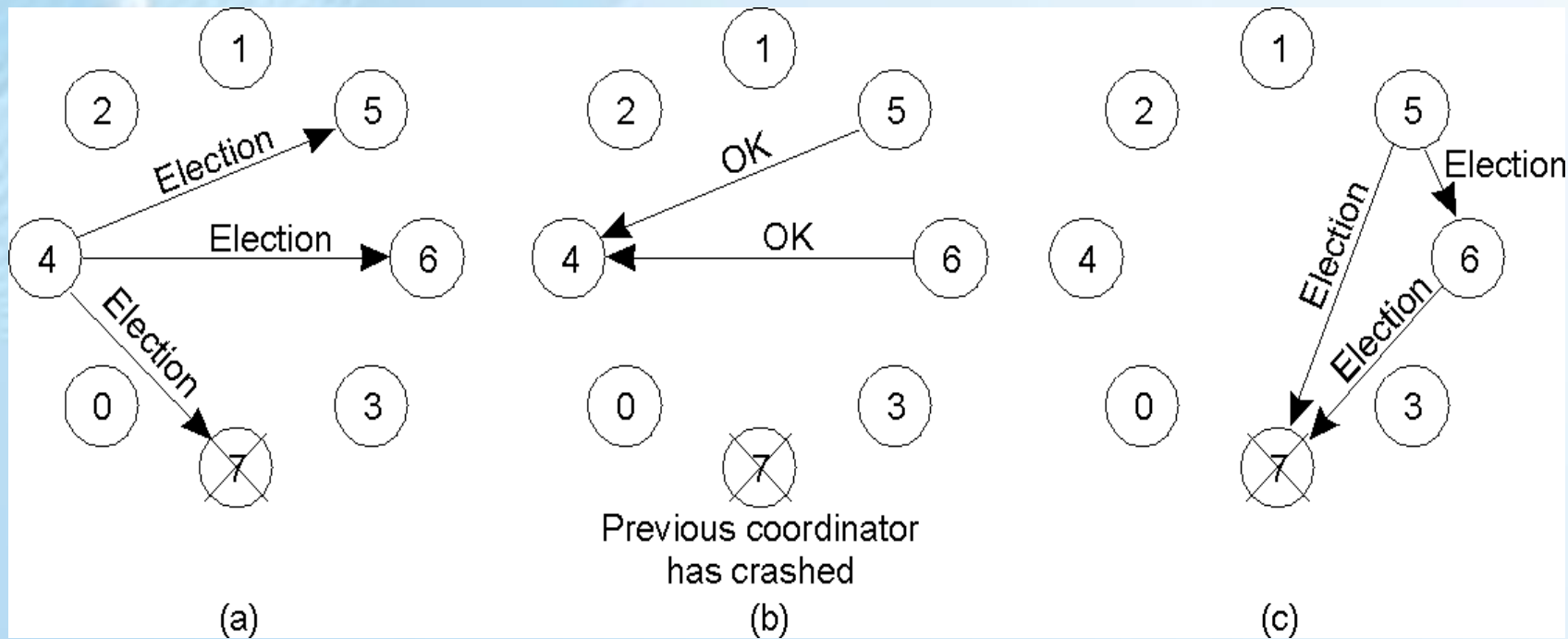
Election Algorithms

- There are situations where one process must act as a **coordinator**, **initiator**, or **perform some special task**.
- **Assumption:**
 - each process has a unique identification number to distinguish them.
 - every process knows **unique identification number of every other** process, but **not the state of each process** (which ones are currently running and which ones are down)
 - The process with highest priority among the currently active processes will be elected as a coordinator.
 - If a failed process recovers, then it can join the set of active processes.
- There is two algorithms: - **Bully algorithm** and
- **Ring algorithm**

Election Algorithms

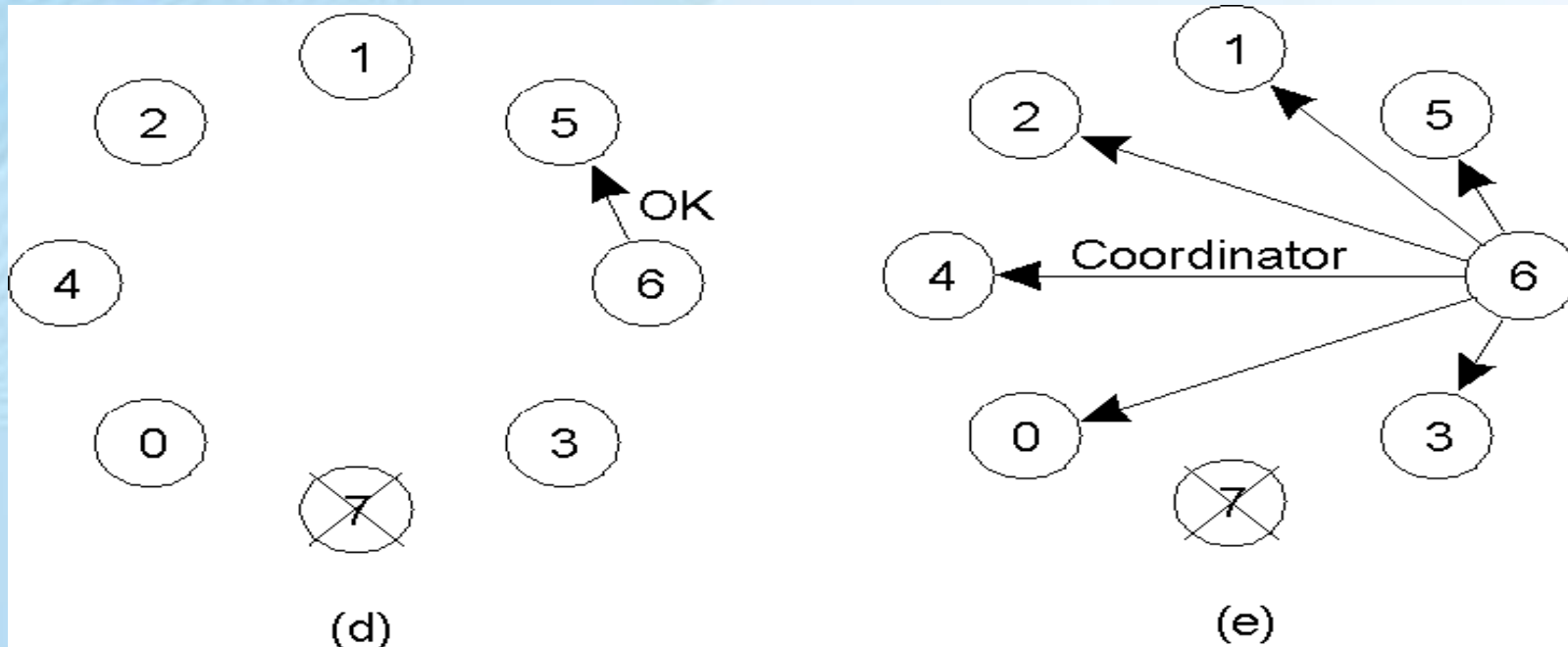
- **The Bully Algorithm** (the biggest person wins)
 - when a process (say P4) notices that the coordinator is no longer responding to requests, it initiates an election as follows:
 1. P4 sends an ELECTION message to all processes with higher numbers (P5, P6, P7).
 - if a process gets an ELECTION message from one of its lower-numbered colleagues, it sends an OK message to the sender and holds an election.
 2. if no one responds, P4 wins the election and becomes a coordinator.
 3. if one of the higher-ups answers, this new process takes over.

Election Algorithms



- a) Process 4 holds an election
- b) Processes 5 and 6 respond, telling 4 to stop
- c) Now 5 and 6 each hold an election

Election Algorithms



d) Process 6 tells 5 to stop

e) Process 6 wins and tells everyone

- the last one will send a message to all processes telling them that it is a boss
- if a process that was previously down comes back, **it holds an election**

Election Algorithms

- When a process gets an **election message** from one of its lower numbered processes:
- Receiver send an **ok** message back to the sender to indicate that he is alive and will take over.
- Eventually all processes give up apart from one and that one is the new coordinator.
- The new **coordinator** announces its victory by sending all processes a **co-Ordinator message** telling them that it is the new coordinator.

Election Algorithms



- If a process that was previously down comes back?
 - It held an election.
 - If it happens to be the highest process currently running, it will win the election and take over the coordinators job.
- “biggest guy” always wins and hence “bully” algorithm.

Election Algorithms

■ The Ring Algorithm

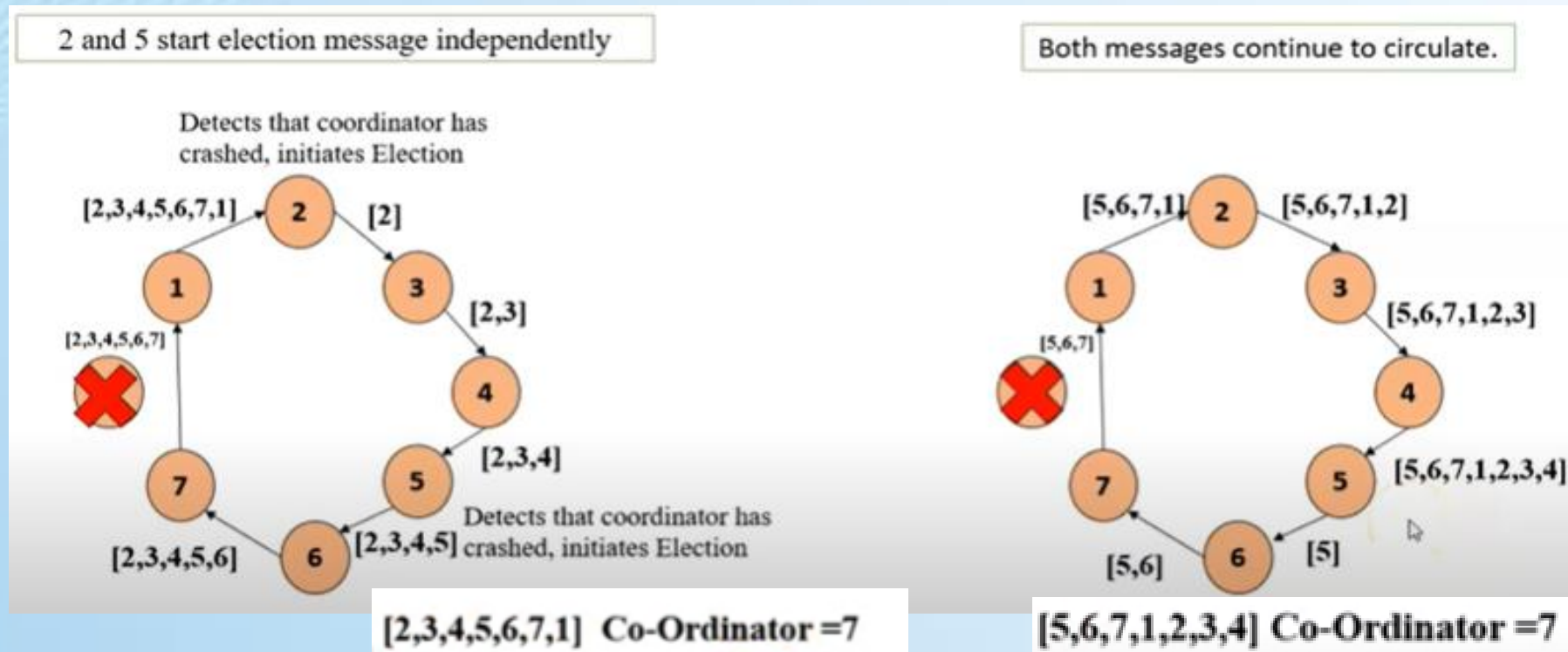
- based on the use of a ring
- assume that processes are physically or logically ordered, so that each process knows **who its successor is**.
- when a process notices that the coordinator is **not functioning**,
 - it builds an ELECTION message containing its own process number and sends it to its successor.
 - if the successor is down, **the sender goes to the next number**, or the next until a running process is found.

Election Algorithms

- at each step, the sender **adds its own process number** to the list in the **message making itself a** candidate.
- eventually the message **gets back** to the originating or initiator process; the process with the **highest number** is elected as coordinator, the message type is changed to **coordinator** and **circulated once again to announce** the **coordinator** and **who** the members of the ring are.

Election Algorithms

- e.g., processes 2 and 5 simultaneously discover that the previous coordinator has crashed
- they build an ELECTION message and a coordinator is chosen
- although the process is done twice, there is no problem, only a little bandwidth is consumed



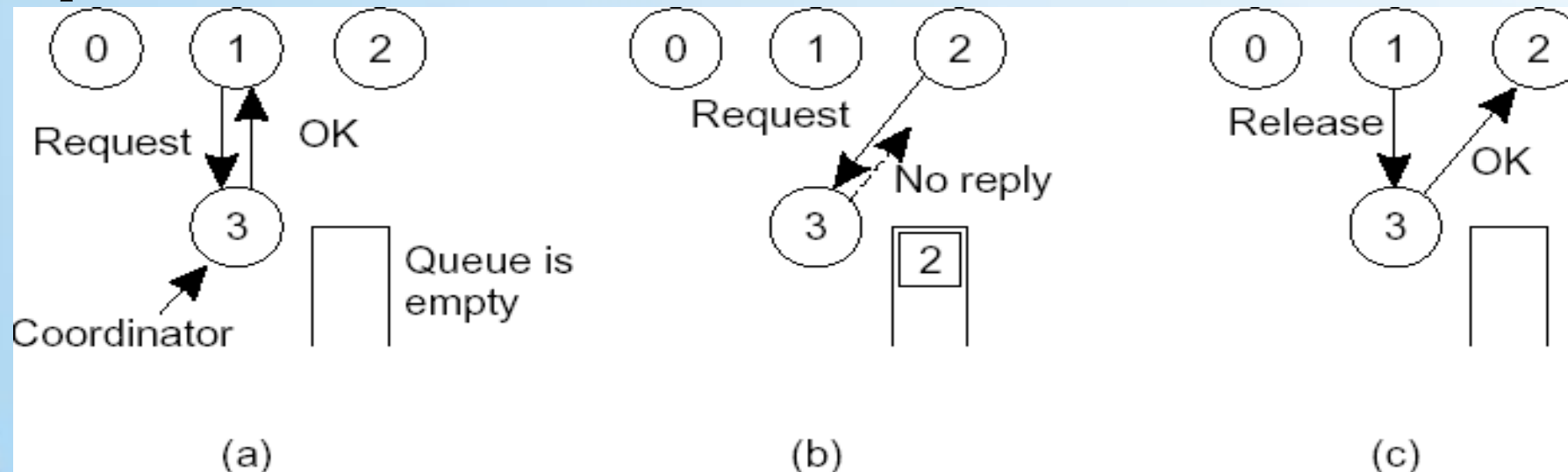
election algorithm using a ring

Mutual Exclusion

- when a process has to read or update shared data (quite often to use a shared resource such as a printer, a file, etc.), it first enters a **critical region** to achieve **mutual exclusion**.
- in single processor systems, critical regions are protected using **semaphores**, **monitors**, and similar constructs.
- how are critical regions and mutual exclusion implemented in **distributed systems**?
- three algorithms: **centralized**, **distributed**, and **token ring**
- **A Centralized Algorithm**
 - a coordinator is appointed and is in charge of granting permissions
 - three messages required: **request**, **grant**, **release**

Cont'd

- a) process 1 asks the coordinator for permission to enter a critical region; permission is granted.
- b) process 2 then asks permission to enter the same critical region; the coordinator **does not reply** (could also send a no message; is implementation dependent), but **queues** process 2; process 2 is blocked.
- c) when process 1 exits the critical region, it tells the coordinator, which then replies to 2.



Cont'd

- the algorithm
 - guarantees mutual exclusion
 - is fair - first come first served
 - no starvation
 - easy to implement; requires only three messages: request, grant, release
- **Drawback:** a failure of the coordinator crashes the system (specially if processes block after sending a request); it also becomes a performance bottleneck.

Cont'd

■ A Distributed Algorithm

- assume that there is a **total ordering of all events** in the system, such as using Lamport timestamps
- when a process wants to enter a critical region it **builds a message** (containing name of **the critical region it wants to enter**, its **process number**, current time) and **sends it to everyone including itself**.
- the sending of a message is assumed to be reliable; i.e., every message is acknowledged.

Cont'd

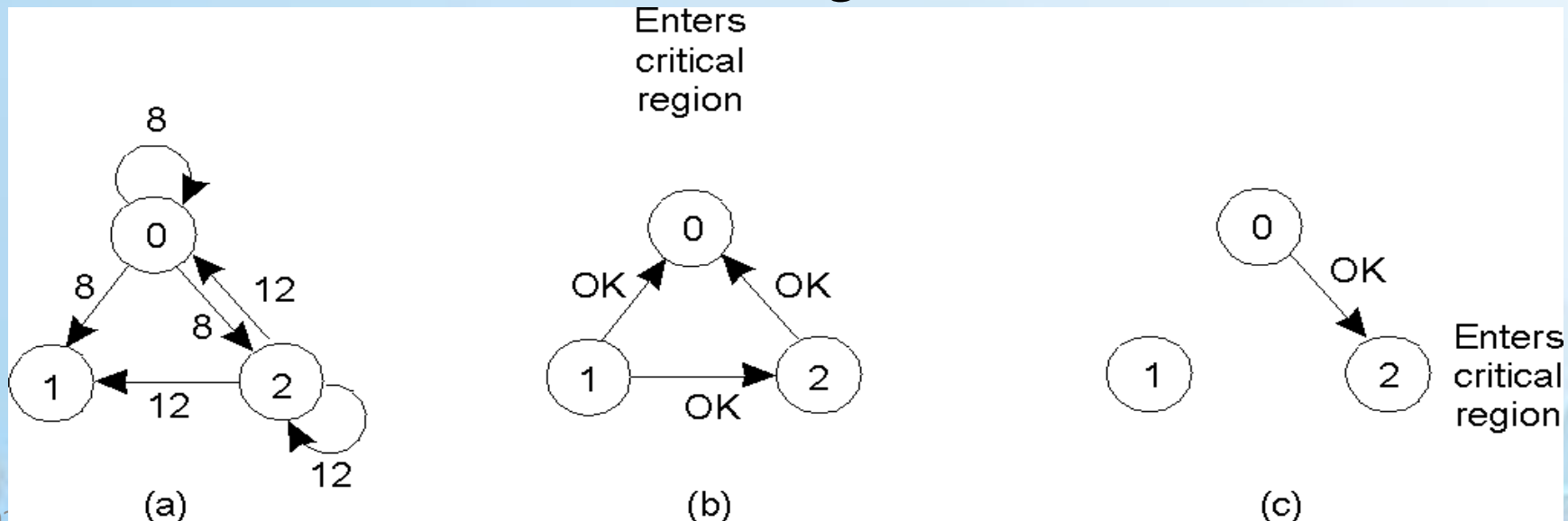


- When a process receives a request message from another process, the action it takes depends on its state with respect to the critical region named in the message.
- Three cases have to be distinguished:
 1. if the receiver **is not in a critical region** and **does not want to enter it**, it sends back an OK message to the sender.
 2. if the receiver is already in the critical region, **it does not reply**; instead it **queues the request**.
 3. if the receiver wants to enter the critical region but has not yet done so, **it compares the timestamp of the message it sent with the incoming one**; the **lowest** wins; if the incoming message is lower, it sends an OK message; otherwise it queues the incoming message and does not do anything.
- When the sender gets a reply from all processes, it may enter into the critical region
- When it finishes it sends an OK message to **all processes in its queue**.

Cont'd



- Is it possible that two processes can enter into the critical region at the same time if they initiate a message at the same time? NO
 - Two processes (0 and 2) want to enter the same critical region at the same moment
 - Process 0 has the lowest timestamp, so it wins
 - When process 0 is done, it sends an OK message, so process 2 can now enter into its critical region



Cont'd



- Mutual exclusion is guaranteed without deadlock or starvation.
- The total number of messages required per entry is now $2(n-1)$, where n is the total number of processes in the system.
- **No single point of failure exists**; unfortunately n points of failure
- Hence, it is slower, more complicated, more expensive, and less robust than the previous one; but shows that a distributed algorithm is possible.
- **Problem:**
 1. The single point of failure has been replaced by n point of failure.
 2. If any process crashes, it will fail to respond to request.

Cont'd

3. This silence will be interpreted (incorrectly) as denial of permission, thus blocking all subsequent attempts by all processes to enter all critical regions.

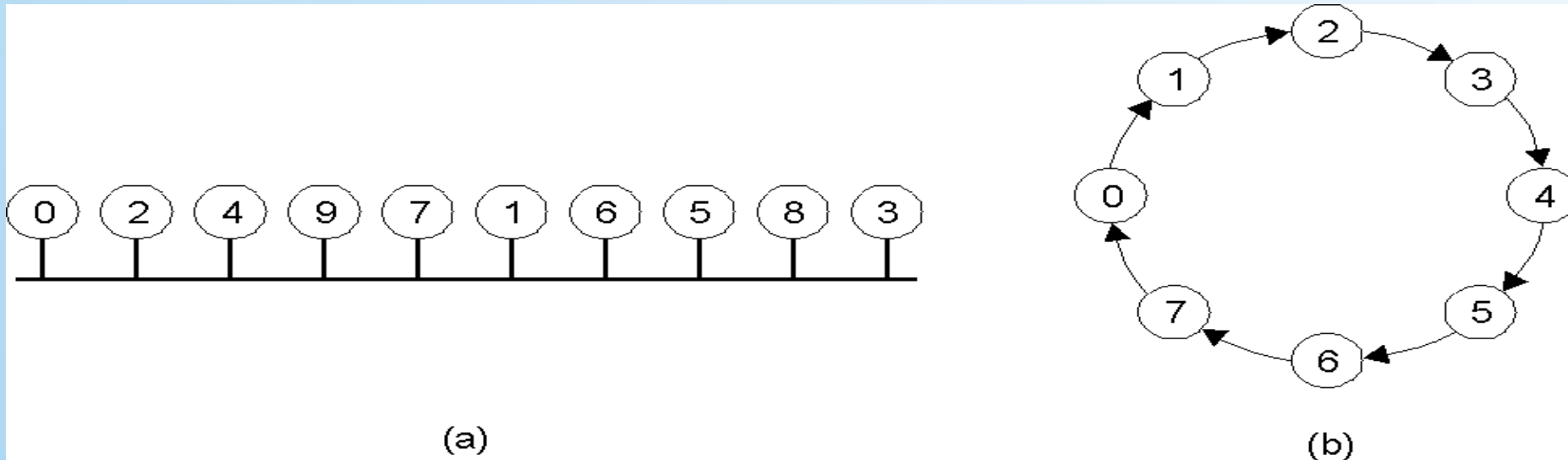
Solution

- When a request comes in, the receiver always sends a reply, either granting or denying permission.
- Whenever either a request or a reply is lost, the sender times out and keeps trying until either a reply comes back or the sender concludes that the destination is dead.
- After a request is denied, the sender should block waiting for a subsequent OK message.

Cont'd

■ A Token Ring Algorithm

- Here we have a bus network (e.g., Ethernet); with no inherent ordering of the processes. A logical ring is constructed in which each process is assigned a position in the ring. The ring position may be allocated in numerical order of network addresses or some other mean. Each process knows who is next in line after itself.



- a) an unordered group of processes on a network
b) a logical ring constructed in software

Cont'd



- when the ring is initialized, process 0 is given a **token**
- the token **circulates** around the ring
- When a process acquires the token from its neighbor, it checks to see if it is attempting to enter a critical region. If so, the process enters the region, does all the work it needs to, and leaves the region.
- **After it has exited, it passes the token along the ring.**
- it should **not** enter into a second critical region with the same token
- mutual exclusion is guaranteed
- no starvation

Cont'd

- **problems**
 - if the token is lost, it has to be generated, but detecting that it is lost is difficult.
 - if a process crashes
 - can be modified to include acknowledging a token so that the token can bypass a crashed process.
 - in this case every process has to maintain the current ring configuration

Cont'd

- A comparison of the three algorithms (assumes only point-to-point communication channels are used).

<i>Algorithm</i>	<i>Messages per entry/exit</i>	<i>Delay before entry (in message times)</i>	<i>Problems</i>
Centralized	3 - efficient	2	Coordinator crash
Distributed	$2 (n - 1)$	$2 (n - 1)$	Crash of any process
Token ring	1 to ∞	0 to $n - 1$	Lost token, process crash

Thank You!!

